

Báo cáo rà soát test code

<!-- framework: pytest | run_id: run_20260618_023835_966473 -->

Framework: pytest | Run ID: run_20260618_023835_966473

Tóm tắt kỹ thuật

- Framework: pytest
- Kết quả chạy: 9 passed, 100 % coverage của source_under_test.py
- Kiểm tra: Các test được sinh ra kiểm thử các hàm factorial, clamp, safe_divide trong math_rules.py.
- Yêu cầu thực tế: Kiểm thử các tiện ích của fixture pytester (tạo file cấu hình, file python, file text, ...).
- Lỗi blocker quan trọng nhất: Test không liên quan tới yêu cầu – chúng kiểm thử sai module, không kiểm tra bất kỳ hàm nào của pytester được mô tả trong Requirement JSON và Test Plan JSON.

Lỗi nghiêm trọng

- Bằng chứng:
- Yêu cầu: “Create conftest.py file with given source”, “makepyfile”, “maketxtfile”, ... (Requirement JSON).
- Test plan: các TC-001...TC-009 đều mô tả việc gọi pytester.makeconfctest, pytester.makeini, pytester.maketoml, pytester.makepyfile, ... (Test Plan JSON).
- Mã kiểm thử sinh ra chỉ import factorial, clamp, safe_divide từ source_under_test và không hề sử dụng fixture pytester hay các hàm make*. (Mã kiểm thử đã sinh).
- Ảnh hưởng: Các test không kiểm tra bất kỳ hành vi nào được yêu cầu, do đó không đáp ứng mục tiêu kiểm thử và làm sai lệch kết quả “passed”.
- Cách sửa: Viết lại các test để:
 1. Nhận fixture pytester (ví dụ def test_makeconfctest(pytester): ...).
 2. Gọi đúng phương thức pytester.makeconfctest(source), pytester.makeini(source), pytester.maketoml(source), pytester.makepyfile(*args), pytester.makepyfile(kwargs).
 3. Kiểm tra path trả về là pathlib.Path, file tồn tại, nội dung khớp source, và phần mở rộng đúng.

Test còn thiếu

- Bằng chứng:
- Các trường hợp negative, boundary, exception (TC-007, TC-008) được liệt kê trong Test Plan nhưng không có trong mã test sinh ra.
- Ảnh hưởng: Không có kiểm thử cho:
 - ValueError khi source rỗng (TC-007).
 - Hành vi khi tạo file đã tồn tại (TC-008).
- Cách sửa: Thêm test:

```
def test_makeconfctest_empty_source(pytester):  
    with pytest.raises(ValueError):  
        pytester.makeconfctest("")
```

và các test cho trường hợp trùng tên file, xác nhận hành vi (overwrite hoặc raise) dựa trên tài liệu thực tế.

Assertion yếu

- Bằng chứng: Các test hiện tại chỉ có các assert so sánh giá trị trả về của hàm toán học với kết quả mong đợi – đây là assert hợp lệ cho chức năng đó, nhưng không liên quan tới yêu cầu.
- Ảnh hưởng: Không có assertion nào kiểm tra các tiêu chí yêu cầu (tồn tại file, nội dung file, phần mở rộng).
- Cách sửa: Thêm các assertion như:
`assert path.exists()`
`assert path.read_text() == source`
`assert path.suffix == ".py" # hoặc .ini, .toml, .txt tùy trường hợp`

Rủi ro maintainability

- Bằng chứng: Các test hiện tại phụ thuộc vào implementation của `factorial`, `clamp`, `safe_divide` – không phải mục tiêu. Khi yêu cầu thay đổi (đối với `pytest`), toàn bộ bộ test sẽ cần viết lại.
- Ảnh hưởng: Dễ gây nhầm lẫn, tăng chi phí bảo trì, và có khả năng bỏ sót các kiểm thử quan trọng của fixture.
- Cách sửa: Định hướng viết test chỉ cho API `pytest` được mô tả trong Requirement, sử dụng fixture chuẩn của `pytest` để tạo môi trường tạm thời, tránh hard-code đường dẫn.

Gợi ý sửa ngay

1. Xóa toàn bộ file test hiện tại (`test_generated.py`) vì không đáp ứng yêu cầu.
2. Tạo file test mới (ví dụ `test_pytester_utils.py`) với nội dung:

```
import pathlib
import pytest
```

```
def test_makeconftest(pytester):
    source = "def foo(): pass"
    path = pytester.makeconftest(source)
    assert isinstance(path, pathlib.Path)
    assert path.name == "conftest.py"
    assert path.read_text() == source
```

```
def test_makeini(pytester):
    source = "[tox]\nenvlist=py39"
    path = pytester.makeini(source)
    assert path.name == "tox.ini"
    assert path.read_text() == source
```

... tương tự cho `maketoml`, `makepyfile` (positional & keyword), `maketxtfile` ...

```
def test_makeconftest_empty_source(pytester):
    with pytest.raises(ValueError):
        pytester.makeconftest("")
```

3. Thêm test cho trường hợp trùng tên file dựa trên hành vi thực tế (overwrite hoặc raise).
4. Chạy lại để xác nhận tất cả các test đều pass và coverage vẫn đạt 100 % cho `source_under_test.py`.

Nếu thực hiện các bước trên, bộ test sẽ đáp ứng đầy đủ yêu cầu, bao phủ các kịch bản dương, âm, và biên, đồng thời loại bỏ các lỗi nghiêm trọng hiện tại.